

Token Replacement in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

Token Replacement in ASP.NET Core Web API

In this article, I will discuss **Token Replacement in ASP.NET Core Web API** with Examples. Please read our previous article, which discusses [how to Set Up Multiple URLs for a Single Resource](#) in the ASP.NET Core Web API Application.

In ASP.NET Core, **Attribute Routing** allows you to define routes directly above controller and action methods using attributes like [Route], [HttpGet], [HttpPost], etc. To make these routes more flexible and dynamic, ASP.NET Core provides **Token Replacement**, a system that automatically substitutes placeholders (tokens) like [controller] and [action] with real values at runtime.

What are Tokens?

Tokens are **placeholders enclosed in square brackets []** used within route templates.

At runtime, ASP.NET Core automatically replaces these tokens with corresponding names.

- [controller] → replaced by the controller name (without the “Controller” suffix).
- [action] → replaced by the action method name.

Why Use Token Replacement?

Token replacement simplifies route management by:

- Reduces hardcoding and repetition in route templates.
- Automatically updates routes if controller or action names change.
- Maintaining a consistent URL structure automatically.
- Makes code cleaner, easier to maintain, and less error-prone.

Real-time Analogy:

- Imagine filling out a **template letter** with placeholders like [Name], [Address], and [Date]. When you send the letter, your software automatically replaces those placeholders with the actual person's name, address, and date.
- In the same way, ASP.NET Core uses tokens like [controller] and [action], which are placeholders that the framework automatically fills in at runtime with the actual controller and action names.

Example Without Using Token Replacement in ASP.NET Core Web API

Before understanding the need and use of token replacement, let us first consider an example without it. Suppose we have two resources in our Employee Controller, and we want to access them using the **controller/action method name**. Then, we can do the same without token replacement, as shown in the code below.

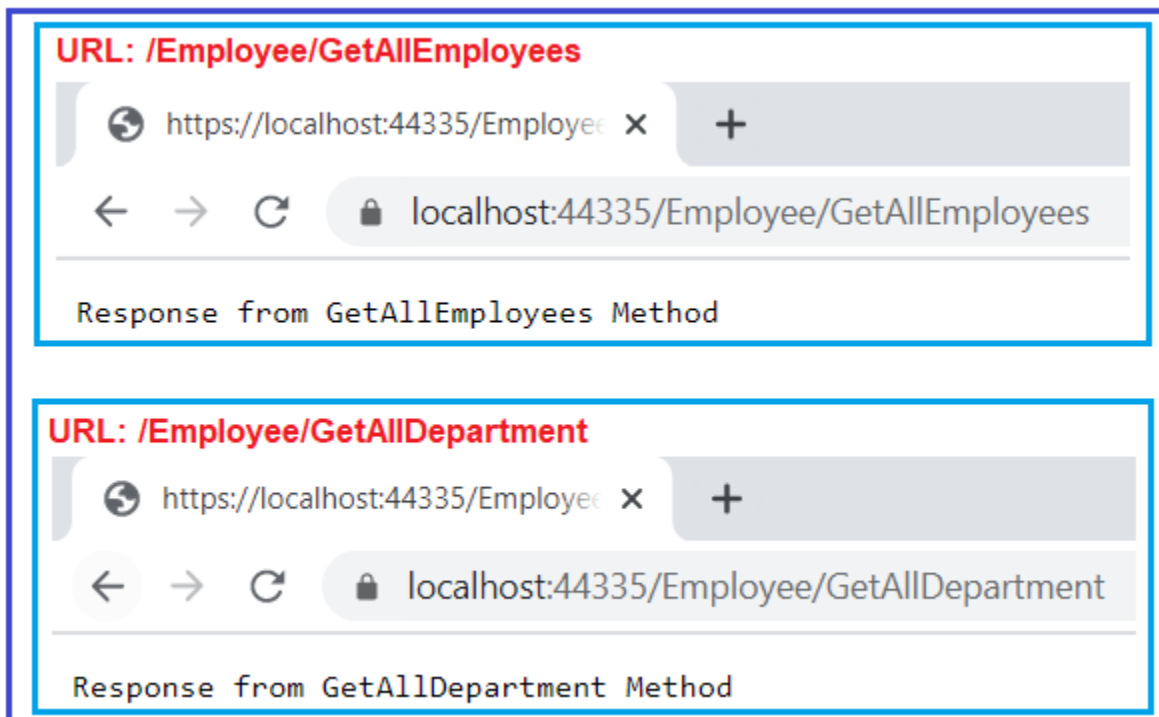
```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [Route("Employee/GetAllEmployees")]
        [HttpGet]
        public string GetAllEmployees()
        {
            return "Response from GetAllEmployees Method";
        }

        [Route("Employee/GetAllDepartment")]
        [HttpGet]
        public string GetAllDepartment()
        {
            return "Response from GetAllDepartment Method";
        }
    }
}
```

Code Explanation:

- Each route is hardcoded.
- You explicitly specify Employee in every [Route] attribute.
- If the controller name changes (e.g., to StaffController), all route strings must be updated manually.
- This approach increases the chance of human error and maintenance effort.

Now run the application, and you can access both the resource using the controller and action method name, as shown in the image below.



Drawback: If you rename the controller or actions later, your routes **break** because the string “Employee” or the action method name is manually written everywhere. This approach works, but is not scalable or maintainable.

Real-time Analogy:

- Think of this like labeling all your office files **manually** with full names every time, e.g., “Employee_Reports_2025”, “Employee_Attendance_2025”.
- If your department name changes from *Employee* to *Staff*, you’ll have to rename every single file manually.
- Similarly, in your code, if you hardcode routes like `Employee/GetAllEmployees`, you will have to edit all those strings whenever the controller name changes. It’s tedious and prone to mistakes.

Example Using Token Replacement in ASP.NET Core Web API

Let us understand how the token replacement works in the ASP.NET Core Web API with an example. Please modify the Employee Controller class as shown below. As you can see, we are applying the token **[controller]** to the `EmployeeController` and the token **[action]** to all its action methods.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class EmployeeController : ControllerBase
    {
        [Route("[action]")]
        [HttpGet]
```

```
public string GetAllEmployees()
{
    return "Response from GetAllEmployees Method";
}

[Route("[action]")]
[HttpGet]
public string GetAllDepartment()
{
    return "Response from GetAllDepartment Method";
}
}
```

What Happens at Runtime?

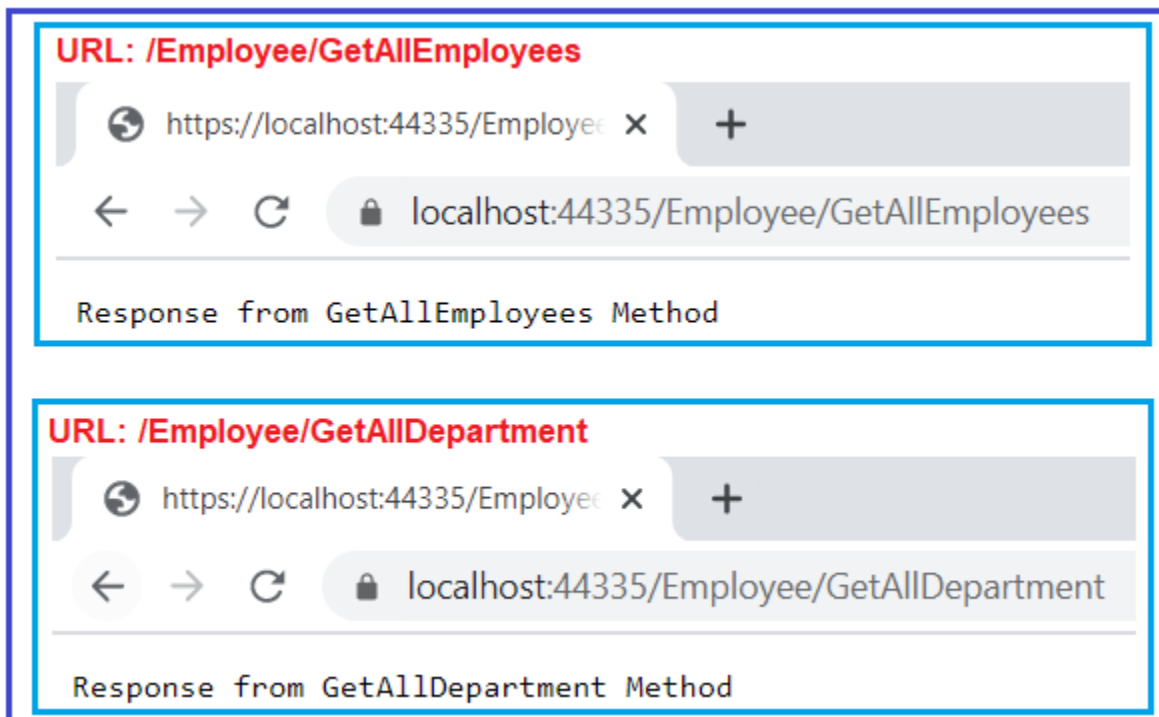
Now, at runtime:

- [controller] → replaced by Employee
- [action] → replaced by method names (GetAllEmployees, GetAllDepartment)

So, the URLs become:

- /Employee/GetAllEmployees
- /Employee/GetAllDepartment

Now, with the above changes in place, you can access the **GetAllEmployees** method with the URL: **/Employee/GetAllEmployees** and the **GetAllDepartment** method with the URL: **/Employee/GetAllDepartment**, as shown in the image below.



Real-time Analogy:

- This is like using **automatic file naming rules** in your system, e.g., your software automatically names each report as [Department]/[ReportType].
- If your department changes from *Employee* to *Staff*, all file names update automatically.
- Likewise, [controller] and [action] dynamically adjust route names; no need to hardcode or manually update them.

Do we need to write the action token on each action method?

Not necessarily. If you want the [action] token to apply to **all action methods** automatically, you can declare it once at the **controller level** instead of repeating it on every method.

Let us understand this with an example. Please modify the Employee Controller class as shown in the code below. As you can see in the code below, we have removed the **[Route("[action]")]** attribute from the action method and modified the Route attribute as **[Route("[controller]/[action]")]**, which is applied at the controller level.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("[controller]/[action]")]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public string GetAllEmployees()
    }
}
```

```

    {
        return "Response from GetAllEmployees Method";
    }

    [HttpGet]
    public string GetAllDepartment()
    {
        return "Response from GetAllDepartment Method";
    }
}

```

What Happens Here:

- The controller-level route template [controller]/[action] applies to **all** action methods.
- ASP.NET Core automatically replaces [controller] and [action] per method call.
- This means both methods inherit the same routing pattern without needing explicit [Route] attributes.

Output URLs:

- /Employee/GetAllEmployees
- /Employee/GetAllDepartment

If tomorrow you add another method `GetEmployeeById(int id)`, it will automatically map to `/Employee/GetEmployeeById/{id}` once you define parameters, no extra configuration required. This approach is **cleaner and scalable** when you have many methods following the same URL pattern.

Now, with the above changes in place, you can also access the **GetAllEmployees** method with the URL: **/Employee/GetAllEmployees** and the **GetAllDepartment** method with the URL: **/Employee/GetAllDepartment**, as shown in the image below.

URL: /Employee/GetAllEmployees

https://localhost:44335/Employee x +

localhost:44335/Employee/GetAllEmployees

Response from GetAllEmployees Method

URL: /Employee/GetAllDepartment

https://localhost:44335/Employee x +

localhost:44335/Employee/GetAllDepartment

Response from GetAllDepartment Method

Real-time Analogy:

- It's like having a **template for all your letters**; you define your company header once, and every letter automatically includes it.
- Instead of writing [action] on every method (every letter), you define the route pattern [controller]/[action] once at the top (letterhead). Now, all methods (letters) automatically follow that pattern without you repeating it.

Controller and Action token applied to the action method

You can also apply [controller]/[action] **directly on each method** instead of at the controller level. In the example below, we apply the Route Attribute only at the action method level, and it works as expected.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [Route("[controller]/[action]")]
        [HttpGet]
        public string GetAllEmployees()
        {
            return "Response from GetAllEmployees Method";
        }

        [Route("[controller]/[action]")]
```

```

[HttpGet]
public string GetAllDepartment()
{
    return "Response from GetAllDepartment Method";
}

[Route("[controller]/[action]/{Id}")]
[HttpGet]
public string GetEmployeeById(int Id)
{
    return $"Response from GetEmployeeById Method, Id : {Id}";
}
}
}

```

Code Explanation:

- Here, both [controller] and [action] tokens are used at the **method level**.
- The {Id} parameter in the third route demonstrates how you can combine tokens with **route parameters**.
- The framework still performs automatic replacement:
 - /Employee/GetAllEmployees
 - /Employee/GetAllDepartment
 - /Employee/GetEmployeeById/1

When to Use:

- This approach is useful when **some methods follow a different URL pattern**.
- You have complete control over each route individually.

Real-time Analogy:

- Think of each **store in a shopping mall**, even though all stores belong to the same mall (controller), each store has its **own signboard and entrance** (method-level route).
- By applying [controller]/[action] directly on each action method, you allow each one to define its own unique route while still following the same naming convention.
- The {Id} parameter works like a customer ticket; you can pass it to access a specific service (e.g., /Employee/GetEmployeeById/1).

Token replacement in ASP.NET Core Web API makes your routing **dynamic, DRY (Don't Repeat Yourself), and easy to maintain**.

By using tokens like [controller] and [action], the framework automatically generates routes that stay in sync with your class and method names. This approach eliminates redundancy, prevents errors when renaming, and keeps

your codebase clean and professional. In short, **token replacement = smarter, cleaner, future-proof routing.**

In the next article, I will discuss [Route Prefix in ASP.NET Core Web API](#) with Examples. In this article, I try to explain token replacement in ASP.NET Core Web API Attribute Routing with examples. I hope you enjoy this Token Replacement in the ASP.NET Core Web API Attribute Routing article.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information